LLDD BE - API Attachment Sales and Timeline

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจทาน และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน
และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. Overview

รายการ	รายละเอียด
Track	BE
Estimate	34 ชั่วโมง = implementation 26 + unit test 8 (30%)
Owner	Peerakorn <Pete> Sakunkaewphithak
Target repository	`SBP/srm-sps-spsap-store-backend` (NestJS + TypeORM · schema `sps_store`) + `SBP/srm-sps-spsap-sbp-bff` (forward ผ่าน client service · ไม่มี DB) สำหรับเส้นที่ FE เรียก
Objective	ออกแบบ APIs สำหรับไฟล์แนบ ข้อมูลยอดขายเพิ่มเติม และ timeline/history

Common contract reference: ทุกหัวข้อ API/FE ต้องยึด LLDD-BE-API-Common-Contracts.pdf และ LLDD-FE-Integration-Contracts.pdf สำหรับ error/auth/format/pagination/action/RBAC ก่อนลงรายละเอียดเฉพาะหน้าหรือเฉพาะ endpoint

1.1 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
ใช้ endpoint ของ	LLDD-BE-API-Document-Workflow-Actions.pdf	`GET /api/v1/sgi/document/{docNo}/timeline`
ถูกเรียกจาก	LLDD-BE-API-Document-Workflow-Actions.pdf	`GET /api/v1/sgi/document/{docNo}/timeline`
ถูกเรียกจาก	LLDD-FE-Document-Detail.pdf	`POST /api/v1/sgi/document/{docNo}/attachments`
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope `{success, data}` · error code · pagination · รูปแบบวันที่/เลขเอกสาร
โครงสร้างข้อมูล	LLDD-BE-Database-Structure.pdf	DDL ของตารางที่หัวข้อ Reference DB Mapping อ้างถึง
workflow engine	LLDD-BE-Workflow-Engine-Definition.pdf	นิยาม state/route/event ที่หัวข้อ Workflow Trigger Event Contract เรียกใช้
แพลตฟอร์มระบบเดิม	LLDD-BE-Integration-SBP-Platform.pdf	header จาก BFF (`x-api-key` / `x-user-*`) · การ reuse ตารางและ service ของระบบ SBP เดิม
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-API-Common-Contracts.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-Database-Structure.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Screen / Functional Scope

- Attachment metadata
- Upload/download adapter
- Sales 4 windows
- Timeline query
- File validation

3. Screenshot Reference

ไม่มีภาพหน้าจอสำหรับหัวข้อนี้ — เป็นเอกสารฝั่ง Backend/Batch ที่ไม่มี UI (ภาพหน้าจอทั้งหมดอยู่ในเอกสารชุด FE)

4. Implementation Flow & Sequence Diagram (Reference)

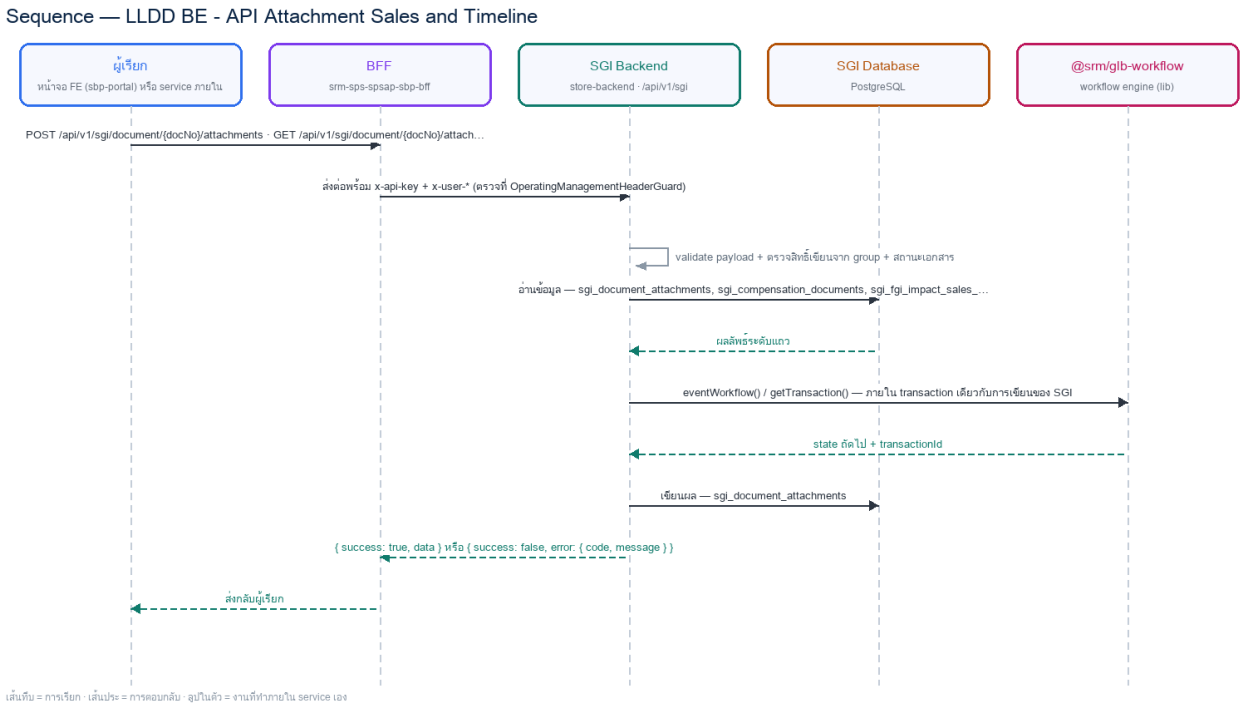
4.1 Implementation Flow (ลำดับขั้นการทำงาน)



รูปที่ 1: Implementation flow reference: LLDD BE - API Attachment Sales and Timeline

4.2 Sequence Diagram (ใครคุยกับใคร ลำดับไหน)

ผู้แสดงและลำดับข้อความในภาพนี้สร้างจาก endpoint ในหัวข้อ 7 และตารางในหัวข้อ Reference DB Mapping ของเอกสารฉบับนี้เอง จึงตรงกับสัญญาเสมอ



รูปที่ 2: Sequence diagram: LLDD BE - API Attachment Sales and Timeline

5. Field, Format, and Validation

Field / UI	Format	Validation	Behavior
docNo	YYYY/xxxxx	required when opening existing document	ใช้ปี ค.ศ. และ running 5 หลัก (มติ 2026-08-06)
storeCode	string 5 digits	numeric length = 5	แสดง leading zero
amount	number, 2 decimals	>= 0	format `#,##0.00` บาท
percent	number, 2 decimals	0-100	ใช้ % และรวม allocation ต้องเท่ากับ 100 — B5: เพิ่ม/ลบร้านที่กระทบเพิ่มเมื่อไรต้องเกลี่ยใหม่ทั้งชุดแล้วคำนวณ `compensateAmount` ของทุกแถวใหม่ ไม่ใช่เฉพาะแถวที่เพิ่ม
sourceSystem	enum	ALLMAP / USER	B5 ที่มาของแถวร้านเปิดใหม่ — `ALLMAP` ระบบ default ให้อัตโนมัติ (Job 9) · USER เจ้าหน้าที่ SBP DSA คีย์เองจากเอกสารแจ้งของหน่วยงานส่งเสริม (ผัง To-Be · SDD สไลด์ 7) · ซ้ำ (`doc_no`, `new_store_code`) ให้คืน `409`
date	DD/MM/YYYY	valid date	payload เป็น ISO ค.ศ. · FE แสดง ค.ศ. เป็นค่าเริ่มต้น (DatePicker buddhistEra=false) แสดง พ.ศ. เฉพาะจุดที่เปิด flag
attachment	file	<= 5 MB	รองรับ vsd, dwg, afp, pdf, mda, zip, wav, mp3, gif, jpg, tif, tiff, htm, html, txt, xml, mpg, mov, ivs, doc, docx, xls, xlsx, pps, ppt, pot, csv
file	multipart	<=5MB	validate extension and content type
sectionCode	string	required on upload	บันทึกว่าแนบในขั้นไหน

5.1 Attachment Storage and Security Design

Attachment API จัดการ binary file จริง ไม่ใช่บันทึกแต่ metadata — แต่ SGI ไม่ได้เป็นเจ้าของ storage layer · ตามมติ DP-8 (ปิด 2026-08-24) SGI เก็บแค่ metadata ใน `sgi_document_attachments` ของตัวเอง แล้ว ยืม service S3 ของระบบ SBP เดิม (`POST /statement/upload-file-aws` · `download-file-aws`) — สิ่งที่ SGI เป็นเจ้าของจริงคือ validation · authorization · metadata · การแปลงเป็น stream ให้ FE

- wrapper ของระบบเดิมเป็น base64 ไม่ใช่ stream (ตรวจ `store-backend` 2026-08-26) — สายส่งจริงคือ FE ← binary stream ← SGI BE ← base64 JSON ← /statement/{upload,download}-file-aws ← S3 · ไฟล์ 5 MB จะกลายเป็น ~6.7 MB ใน JSON (body limit ของ store-backend คือ 100 MB) · ปุ่ม ดาวน์โหลดทั้งหมด (.zip) ห้ามโหลดทุกไฟล์เข้า memory พร้อมกัน ให้ดึงทีละไฟล์แล้ว stream เข้า zip — รายละเอียดเต็มอยู่ที่ LLDD-BE-Integration-SBP-Platform.pdf 5.3

Item	Required value / convention	Developer note
Storage provider	service S3 ของระบบ SBP เดิม (`AwsService` ผ่าน `/statement/{upload,download}-file-aws`)	• มติ DP-8 — SGI ห้ามสร้าง storage adapter/ไม่ต่อ S3 SDK เอง และไม่ต้องเลือก vendor · เก็บ `storage_provider` ไว้เป็น metadata เผื่ออนาคตเท่านั้น
Bucket/container	bucket ของระบบเดิม (ทีม SBP เป็นผู้กำหนด)	ไฟล์ของ SGI ใช้ prefix ของตัวเองใต้ bucket เดียวกับระบบเดิม — lifecycle/backup เป็นของ infra ฝั่งนั้น ไม่ใช่ของ SGI
Object key	`documents/{year}/{docNoSafe}/{attachId}/{sha256Prefix}-{safeFileName}`	`docNoSafe` แทน `/` ด้วย `-`; sanitize filename ก่อนใช้ใน key
Quarantine / AV	ไม่มี (ถอดออก 2026-10-05)	ไม่มีการสแกนไวรัส — ถอดออกทั้งระบบ 2026-10-05 (มติผู้ใช้) เพราะไม่มีตัวสแกนจริงในระบบใดเลย
Allowed extension	vsd, dwg, afp, pdf, mda, zip, wav, mp3, gif, jpg, tif, tiff, htm, html, txt, xml, mpg, mov, ivs, doc, docx, xls, xlsx, pps, ppt, pot, csv	ตรวจทั้ง extension และ content type/magic bytes เท่าที่ platform รองรับ · ไม่อยู่ใน allowlist → 415 `FILE_TYPE_UNSUPPORTED`
การดาวน์โหลด	ไม่มีสถานะสแกนไวรัส (ถอดออก 2026-10-05)	ดาวน์โหลดได้ทุกไฟล์ที่เป็นของเอกสารนั้น — ไม่มีการสแกนไวรัส (ถอดออกทั้งระบบ 2026-10-05 ตามมติผู้ใช้) · ไฟล์ที่ย้ายมาจาก K2 แบบมีแต่ metadata ตอบ 422 `FILE_NOT_MIGRATED` — รายละเอียดเต็มที่หัวข้อ Download Flow
Max size	5 MB ต่อไฟล์	เกินให้คืน 413 `FILE_TOO_LARGE` ก่อน upload เข้า storage
ต้องเลือกไฟล์ก่อน	กดปุ่มแนบเอกสารโดยยังไม่เลือกไฟล์	คืน 422 `ATTACHMENT_FILE_REQUIRED` พร้อมข้อความ verbatim "กรุณาเลือกไฟล์ที่ต้องการแนบก่อนกดแนบเอกสาร" — FE ต้องกันไว้ก่อนยิง API ด้วย

5.2 Attachment Metadata Fields

Field	Meaning	Required behavior
attachId	primary key/identifier	คืนให้ FE หลัง upload
docNo	เลขเอกสาร	attachment ต้อง belong กับ document นี้เท่านั้น
sectionCode	workflow section ตอน upload	บันทึกจาก request และ validate กับ current task/permission
originalFileName	ชื่อไฟล์จากผู้ใช้	เก็บเพื่อแสดงผลและ Content-Disposition
contentType	MIME type	ใช้ร่วมกับ extension validation
fileSizeBytes	ขนาดไฟล์	ต้อง <= 5 MB
storageProvider/bucketName/objectKey	ตำแหน่ง binary	ห้าม expose objectKey ตรงให้ FE
sha256	checksum	ใช้ตรวจ duplicate/corruption
uploadedBy/uploadedAt/deletedFlag	audit metadata	soft delete เท่านั้นเมื่อมีการลบภายหลัง

5.3 Upload Flow

Step	Backend behavior	Error / response
1. Authorize	ตรวจผู้ใช้มีสิทธิ์อ่านเอกสารและ canUploadAttachment/current task owner	ไม่มีสิทธิ์คืน 403
2. Validate multipart	ตรวจ file present, size, extension, content type, sectionCode	คืน 400/413/415 ตาม catalog
3. Hash + ส่งขึ้น storage	คำนวณ sha256 จาก buffer แล้วเรียก `POST /statement/upload-file-aws` (ส่งเป็น base64) เก็บ objectKey ที่ได้กลับมา	service ของระบบเดิม fail คืน 503 และ ไม่ insert metadata
4. Insert metadata	insert `sgi_document_attachments` พร้อม objectKey · sha256 · uploaded_by_name	• ไม่มีขั้น move/promote — wrapper ของระบบเดิมไม่มี API ย้าย object ให้ SGI เรียก
6. Respond	คืน attachId, fileName, fileSizeBytes, uploadedAt	ไม่คืน bucket/objectKey ให้ FE

5.4 Download Flow and Authorization

นโยบายดาวน์โหลดไฟล์แนบ (ปรับ 2026-10-05 · มติผู้ใช้: เอาเรื่องการสแกนไวรัสออก):

- ระบบไม่มีการสแกนไวรัส — เดิมเก็บสถานะ `scan_status` แต่ไม่เคยมีตัวสแกนจริงในระบบใดเลย ทุกไฟล์จึงค้าง "รอตรวจ" และต้องพึ่งสวิตช์ใน `mas_param` เปิดทางให้ดาวน์โหลด · ถอดออกทั้งคอลัมน์ `scan_status` / `scanned_at` / `scan_message` · สวิตช์ใน `mas_param` · error `FILE_SCAN_*` · header `X-SGI-Scan-Status`
- ไฟล์ของเอกสารนั้นดาวน์โหลดได้ทุกไฟล์ (ตรวจสิทธิ์อ่านเอกสาร + ไฟล์ต้องเป็นของ docNo นั้น)
- ที่ยังป้องกันอยู่: รับเฉพาะ 12 นามสกุลที่ไม่ใช่ไฟล์รันได้ · เพดาน 5 MB · ตรวจเนื้อไฟล์ให้ตรงนามสกุลที่ BFF · ดาวน์โหลดด้วย `Content-Disposition: attachment` (เบราว์เซอร์ไม่เปิดไฟล์ให้เอง)
- ไฟล์ที่ย้ายมาจาก K2 แบบมีแต่ metadata (`K2-LEGACY` + ขนาด 0) ตอบ 422 `FILE_NOT_MIGRATED` ไม่ยิง S3

Step	Backend behavior	Error / response
1. Validate path	ตรวจ docNo/attachId และ attachment belongs to docNo	ไม่พบคืน 404
2. Authorize read	สิทธิ์เท่ากับ document read หรือ report/admin ที่ได้รับสิทธิ์	ไม่มีสิทธิ์คืน 403
3. Check row	`deleted_flag = 'N'` และตัดแถว K2-LEGACY ขนาด 0 (FILE_NOT_MIGRATED) (คอลัมน์เป็น CHAR(1) ไม่ใช่ boolean)	ดาวน์โหลดได้ทุกไฟล์ที่เป็นของเอกสารนั้น — ไม่มีการสแกนไวรัส (ถอดออกทั้งระบบ 2026-10-05 ตามมติผู้ใช้) · ไฟล์ที่ย้ายมาจาก K2 แบบมีแต่ metadata ตอบ 422 `FILE_NOT_MIGRATED` — ห้าม hardcode รายการสถานะในโค้ด ให้อ่านสวิตช์จาก `mas_param` ทุกครั้ง
4. Stream	เรียก `POST /statement/download-file-aws` ได้ base64 แล้ว decode เป็น buffer ก่อน stream ออกไป	• ไม่มี signed URL ให้ใช้ — wrapper ของระบบเดิมไม่คืน presigned url · ตั้ง Content-Type และ Content-Disposition จาก metadata
5. Audit	บันทึกร่องรอยการดาวน์โหลดที่ application log (structured)	• ตาราง `audit_logs` ถูกตัดไปแล้ว 2026-08-07 — ห้ามอ้างตารางนี้ · ต้อง trace userId/docNo/attachId/requestId ได้จาก log

5.5 Download Endpoint Contract

Method	Path	Response
GET	/api/v1/sgi/document/{docNo}/attachments/{attachId}/download	binary stream; headers Content-Type, Content-Length, Content-Disposition

5.6 Attachment Repository SQL Reference

```
-- bind ตามลำดับ: $1=docNo · $2=sectionCode · $3=fileName · $4=mimeType · $5=fileSize · $6=storageProvider · $7=bucket · $8=objectKey · $9=sha256 · $10=userId · $11=userFullName · $12=attachId
-- Insert metadata after storage write (ไม่มีการสแกนไวรัส · ถอดออก 2026-10-05)
INSERT INTO sgi_document_attachments (
    doc_no, section_code, file_name, mime_type, file_size,
    storage_provider, bucket, object_key, sha256,
    uploaded_by, uploaded_by_name, uploaded_at, deleted_flag
) VALUES (
    $1 /* docNo */, $2 /* sectionCode */, $3 /* fileName */, $4 /* mimeType */, $5 /* fileSize */,
    $6 /* storageProvider */, $7 /* bucket */, $8 /* objectKey */, $9 /* sha256 */,
    $10 /* userId */, $11 /* userFullName */, CURRENT_TIMESTAMP, 'N'
)
RETURNING attach_id;

-- Load attachment for download. Authorization is checked in service before streaming.
SELECT
    attach_id, doc_no, file_name, mime_type, file_size,
    storage_provider, bucket, object_key, sha256
FROM sgi_document_attachments
WHERE doc_no = $1 /* docNo */
  AND attach_id = $12 /* attachId */
  AND deleted_flag = 'N';
```

5.9 Input / Progress / Output Contract

Stage	Contract for implementation
Input	POST /api/v1/sgi/document/{docNo}/attachments; GET /api/v1/sgi/document/{docNo}/attachments/{attachId}/download; GET /api/v1/sgi/document/{docNo}/attachments/download-all
Progress	Validate docNo/permission; Validate file size/type; Store file metadata; Load sales summary and transactions
Output	sgi_document_attachments

5.90 Endpoint Implementation Contract

Endpoint	Use-case owner	Service/repository behavior	Definition of done
POST /api/v1/sgi/document/{docNo}/attachments	Upload attachment API	Validate docNo/permission	file >5MB returns 413
GET /api/v1/sgi/document/{docNo}/attachments/{attachId}/download	ดาวน์โหลดไฟล์แนบรายไฟล์ผ่าน BE — ตรวจสิทธิ์เอกสาร + attachment ต้องเป็นของ docNo ตามนโยบาย (ดาวน์โหลดได้ทุกไฟล์ที่เป็นของเอกสารนั้น — ไม่มีการสแกนไวรัส (ถอดออกทั้งระบบ 2026-10-05 ตามมติผู้ใช้) · ไฟล์ที่ย้ายมาจาก K2 แบบมีแต metadata ตอบ 422 `FILE_NOT_MIGRATED`)	Validate file size/type	unsupported file type returns 415

Endpoint	Use-case owner	Service/repository behavior	Definition of done
GET /api/v1/sgi/document/{docNo}/attachments/download-all	ดาวน์โหลดไฟล์แนบทั้งหมดเป็น .zip — ไม่มีไฟล์ที่ผ่าน scan เลยตอบ 404 (ไม่คืน zip เปล่า) · ● แก้ 2026-09-30 — ต้องคัดแถว `K2-LEGACY` + `file_size = 0` ออกก่อนยิง S3 แล้วตอบ `FILE_NOT_MIGRATED` (422) เมื่อเอกสารมีแต่แถวแบบนั้น (7,758 เอกสารบน dev) · ทุกแถวล้มที่ storage ตอบ `DOWNLOAD_FROM_STORAGE_FAILED` (502) ไม่ใช่ 404 · zip ที่ไม่ครบต้องบอกจำนวนผ่าน header `X-SGI-Attachment-Total` / `-Included` / `-Skipped-Not-Migrated` / `-Skipped-Failed` (ประกาศใน `Access-Control-Expose-Headers`) — ของเดิม `continue` เงียบ ผู้ใช้เปิด zip แล้วไฟล์ขาดโดยไม่มีอะไรบอก	Store file metadata	sales windows are ordered
GET /api/v1/sgi/document/{docNo}/timeline	Timeline/history API	Load sales summary and transactions	timeline newest/oldest order matches FE expectation

5.91 Backend Execution Sequence

Step	Behavior specific to this LLDD	Failure/test evidence
1	Validate docNo/permission	upload success
2	Validate file size/type	upload too large
3	Store file metadata	download missing file
4	Load sales summary and transactions	sales not found
5	Return timeline ordered by action time	timeline empty

5.92 Workflow Trigger Event Contract

งานชิ้นนี้ ต้องเรียก workflow engine ตามตารางด้านล่าง · ชื่อ function ยึด API 8 ตัวของ `@srm/glb-workflow` ตามชีต `Detail` ของ TSM SRM LLDD SBP workflow ไฟล์1.2 — รายละเอียด signature และตารางที่ engine เขียน ดู LLDD-BE-Workflow-Engine-Definition.pdf หัวข้อ 5.3

จุดที่เรียก (call site)	Engine function	พารามิเตอร์หลัก	กติกา / transaction boundary
แท็บประวัติ (timeline)	`getHistory`	versionId, referenceId	อ่าน `sgi_consideration_logs` ของ SGI เป็น timeline เต็ม — ไม่เรียก getHistory() เพราะ engine ไม่มี

- ● กติกาเหล็ก: ตาราง `sps_store.workflow_*` (13 ตาราง) เป็นของ lib — SGI R เท่านั้น ห้าม INSERT/UPDATE/DELETE ตรงในทุกกรณี
- ทุกการเรียก engine ต้องผ่านตัวห่อกลาง `WorkflowGateway` ที่นิยามใน LLDD-BE-API-Common-Contracts.pdf (timeout · retry · map error เข้า envelope) ห้าม import lib ตรงจาก service
- unit test ต้อง mock engine และครอบอย่างน้อย: เรียกสำเร็จ · engine โยน error แล้ว rollback ฝั่ง SGI ครบ · เรียกซ้ำด้วย referenceId เดิมไม่เกิดผลซ้ำ

6. Button / User Action Mapping

Action	Trigger	API / Service	Expected Result
Upload attachment	POST multipart	attachment.service.upload	store file and metadata
Download attachment	GET	attachment.service.download	stream file
Get sales	GET	sales.service.getDocumentSales	return sales windows

7. API Contract

POST /api/v1/sgi/document/{docNo}/attachments

Upload attachment API

Request

```
{
  "file": "multipart <= 5MB",
  "sectionCode": "06"
}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
file	string	Yes	UTF-8; use value domain described by endpoint purpose
sectionCode	string	Yes	canonical code; do not replace with display label

Response

```
{
  "attachId": 771,
  "fileName": "ไฟล์evidence"
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
attachId	integer	Yes	UTF-8; use value domain described by endpoint purpose
fileName	string	Yes	UTF-8; use value domain described by endpoint purpose

GET /api/v1/sgi/document/{docNo}/attachments/{attachId}/download

ดาวน์โหลดไฟล์แนบรายไฟล์ผ่าน BE — ตรวจสิทธิ์เอกสาร + attachment ต้องเป็นของ docNo ตามนโยบาย (ดาวน์โหลดได้ทุกไฟล์ที่เป็นของเอกสารนั้น — ไม่มีการสแกนไวรัส (ถอดออกทั้งระบบ 2026-10-05 ตามมติผู้ใช้) · ไฟล์ที่ย้ายมาจาก K2 แบบมีแต่ metadata ตอบ 422 `FILE_NOT_MIGRATED`)

Query Params

```
{}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
-	none	No	No fields

Response


```
{
  "contentType": "application/pdf",
  "note": "binary stream · ไฟล์จริงอยู่บน S3 ผ่าน service ของระบบ SBP เดิม"
}
```


Response Field Schema

Field	Type	Required	Constraint / Meaning
contentType	string	Yes	UTF-8; use value domain described by endpoint purpose
note	string	Yes	UTF-8; use value domain described by endpoint purpose

GET /api/v1/sgi/document/{docNo}/attachments/download-all

ดาวน์โหลดไฟล์แนบทั้งหมดเป็น .zip — ไม่มีไฟล์ที่ผ่าน scan เลยตอบ 404 (ไม่คืน zip เปล่า) · ● แก้ 2026-09-30 — ต้องคัดแถว `K2-LEGACY` + `file_size = 0` ออกก่อนยิง S3 แล้วตอบ `FILE_NOT_MIGRATED` (422) เมื่อเอกสารมีแต่แถวแบบนั้น (7,758 เอกสารบน dev) · ทุกแถวล้มที่ storage ตอบ `DOWNLOAD_FROM_STORAGE_FAILED` (502) ไม่ใช่ 404 · zip ที่ไม่ครบต้องบอกจำนวนผ่าน header `X-SGI-Attachment-Total` / `-Included` / `-Skipped-Not-Migrated` / `-Skipped-Failed` (ประกาศใน `Access-Control-Expose-Headers`) — ของเดิม `continue` เงียบ ผู้ใช้เปิด zip แล้วไฟล์ขาดโดยไม่มีอะไรบอก

Query Params


```
{}
```


Request Field Schema

Field	Type	Required	Constraint / Meaning
-	none	No	No fields

Response


```
{
  "contentType": "application/zip",
  "fileName": "2026-00123-attachments.zip"
}
```


Response Field Schema

Field	Type	Required	Constraint / Meaning
contentType	string	Yes	UTF-8; use value domain described by endpoint purpose
fileName	string	Yes	UTF-8; use value domain described by endpoint purpose

GET /api/v1/sgi/document/{docNo}/timeline

Timeline/history API

Query Params


```
{
  "docNo": "2026/00123"
}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
docNo	string	No	ค.ศ. YYYY/xxxxx

Response


```
{
  "items": [
    {
      "section": "06",
      "result": "ชดเชย"
    }
  ]
}
```


Response Field Schema

Field	Type	Required	Constraint / Meaning
items	array<object>	Yes	JSON array; element type shown in Type column
items[].section	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].result	string	Yes	UTF-8; use value domain described by endpoint purpose

8. Reference DB Mapping (No Database Page Work)

ส่วนนี้เป็นข้อมูลอ้างอิงสำหรับการ implement API/Job เท่านั้น ไม่ใช่งานสร้างหน้า Database, ไม่ใช่งานออกแบบ DB page และไม่ถูกนับเป็น deliverable แยกของ FE/BE

Table / Object	R/W	Usage
sgi_document_attachments	R/W	metadata ไฟล์แนบและ section ที่แนบ
sgi_compensation_documents	R	ตรวจเอกสารและ impact_process_id
sgi_fgi_impact_sales_summaries	R	หัวข้อมูลยอดขาย growth_rate_diff/total_working_days
sgi_sales_transactions	R	ยอดขายรายวัน 4 windows
sgi_consideration_logs	R	timeline/history

9. Skeleton Code (store-backend + BFF)

โครงโค้ดตั้งต้นของเอกสารฉบับนี้ ยึด convention จริงของ `srm-sps-spsap-store-backend` (NestJS 11 + TypeORM, schema `sps_store`, custom provider `DATA_SOURCE` ที่ route SELECT ไป slave pool) และ `srm-sps-spsap-sbp-bff` (ไม่มี DB, forward ผ่าน client service). ทุกจุดที่ต้องเติมกำกับด้วย `// TODO:` และ response ทุกเส้นถูกห่อเป็น `{success, data}` โดย ResponseInterceptor อยู่แล้ว จึงห้าม service ห่อซ้ำ

9.1 ผังไฟล์ที่ต้องสร้าง

Path	หน้าที่
store-backend · src/modules/sgi-attachment-sales-timeline/sgi-attachment-sales-timeline.controller.ts	route ทั้งหมดของเอกสารนี้ (3 เส้น) + `@UseGuards(HttpHeaderGuard)` + `@UserId()`
store-backend · src/modules/sgi-attachment-sales-timeline/sgi-attachment-sales-timeline.service.ts	business logic — inject `'DATA_SOURCE'` แล้วยิง raw SQL, mutation ใช้ QueryRunner transaction

Path	หน้าที่
store-backend · src/modules/sgi-attachment-sales-timeline/sgi-attachment-sales-timeline.sql.ts	เก็บ SQL ต่อ endpoint (คัดจากหัวข้อ 10) แยกออกจาก service ให้ทดสอบ/รีวิวง่าย · คีย์ = ชื่อ handler เช่น `getSgiMasterFactors` · บล็อกที่มีหลาย statement ให้แยกเป็นหลายคีย์ โดยเติมท้ายชื่อให้สื่อความ เช่น DELETE master ที่มี 2 statement → `removeSgiMasterFactorsByCodeInUse` (SELECT ตรวจการใช้งาน) + `removeSgiMasterFactorsByCode` (DELETE)
store-backend · src/modules/sgi-attachment-sales-timeline/dto/sgi-attachment-sales-timeline.dto.ts	DTO + class-validator ตาม validation ในหัวข้อฟิลด์ของเอกสารนี้
store-backend · src/modules/sgi-attachment-sales-timeline/sgi-attachment-sales-timeline.module.ts	ประกอบ controller/service/providers แล้ว register ที่ `app.module.ts`
store-backend · src/entitys/sgi-document-attachments.entity.ts	entity ของ `sgi_document_attachments` (`@Entity({schema: process.env.DB_SCHEMA})`, ไม่ประกาศ relation) — entity ร่วมหลายเอกสาร: ประกาศครั้งเดียวแล้วอ้างอิง อย่าสร้างซ้ำ
store-backend · src/entitys/sgi-compensation-documents.entity.ts	entity ของ `sgi_compensation_documents` (`@Entity({schema: process.env.DB_SCHEMA})`, ไม่ประกาศ relation) — entity ร่วมหลายเอกสาร: ประกาศครั้งเดียวแล้วอ้างอิง อย่าสร้างซ้ำ
store-backend · src/entitys/sgi-fgi-impact-sales-summaries.entity.ts	entity ของ `sgi_fgi_impact_sales_summaries` (`@Entity({schema: process.env.DB_SCHEMA})`, ไม่ประกาศ relation)
store-backend · src/providers/sgi/sgi.ts	repository provider แบบ factory ผูก token string กับ `DATA_SOURCE` — ไฟล์ร่วมของทุกเอกสาร BE ให้ merge array เพิ่ม ห้ามเขียนทับ
store-backend · sql/deploy-sgi-attachment-sales-timeline.sql	DDL production แบบ idempotent (ทีมนี้ไม่ใช้ migration เป็นหลัก)
BFF · src/common/client-services/sgi-client.service.ts	client ต่อจาก `BaseClientService` ตั้ง baseUrl + `x-api-key` ตอน `onModuleInit`
BFF · src/modules/sgi-attachment-sales-timeline/sgi-attachment-sales-timeline.controller.ts	route ฝั่ง BFF prefix /bff/sgi/… + `@UseGuards(AuthGuard('jwt'))`
BFF · src/modules/sgi-attachment-sales-timeline/sgi-attachment-sales-timeline.service.ts	แนบ `x-user-id` / `x-user-group-id` / `x-user-permissions` แล้ว forward ไป backend

เส้นที่ไม่ต้อง implement ใหม่ในเอกสารนี้:

Endpoint	จุดประสงค์	เหตุผล
GET /api/v1/sgi/document/{docNo}/timeline	Timeline/history API	reference — implement ที่เอกสาร `LLDD-BE-API-Document-Workflow-Actions.pdf` (1 เส้น = 1 เจ้าของ ไม่ประกาศ controller ซ้ำ ไม่งั้น NestJS จะ register ทับกันเงียบ ๆ)

9.2 Controller (store-backend)

```
// src/modules/sgi-attachment-sales-timeline/sgi-attachment-sales-timeline.controller.ts
import { Body, Controller, Get, Param, Post, UseGuards } from '@nestjs/common';
import { HttpHeaderGuard } from '../../guards/http-header.guard';
import { UserId } from '../../common/decorators/user-id.decorator';
import { SgiAttachmentSalesTimelineService } from './sgi-attachment-sales-timeline.service';
import { CreateSgiDocumentAttachmentsBodyDto } from './dto/sgi-attachment-sales-timeline.dto';

// LLDD BE - API Attachment Sales and Timeline
// BFF เรียกด้วย x-api-key และแนบ x-user-id / x-user-group-id / x-user-permissions มาให้
@Controller('document/:docNo/attachments')
@UseGuards(HttpHeaderGuard)
export class SgiAttachmentSalesTimelineController {
  constructor(private readonly service: SgiAttachmentSalesTimelineService) {}

  // POST /api/v1/sgi/document/{docNo}/attachments — Upload attachment API
  @Post()
  createSgiDocumentAttachments(
    @Param('docNo') docNo: string,
    @Body() body: CreateSgiDocumentAttachmentsBodyDto,
    @UserId() userId: string,
  ) {
    // TODO: ตรวจ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.createSgiDocumentAttachments(docNo, body, userId);
  }
```

```
  // GET /api/v1/sgi/document/{docNo}/attachments/{attachId}/download — ดาวน์โหลดไฟล์แนบรายไฟล์ผ่าน BE — ตรวจสิทธิ์เอกสาร + attachment ต้องเป…
  @Get(':attachId/download')
  getSgiDocumentAttachmentsDownload(
    @Param('docNo') docNo: string,
    @Param('attachId') attachId: string,
    @UserId() userId: string,
  ) {
    // TODO: ตรวจ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.getSgiDocumentAttachmentsDownload(docNo, attachId, userId);
  }

  // GET /api/v1/sgi/document/{docNo}/attachments/download-all — ดาวน์โหลดไฟล์แนบทั้งหมดเป็น .zip — ไม่มีไฟล์ที่ผ่าน scan เลยตอบ 404 (…
  @Get('download-all')
  getSgiDocumentAttachmentsDownloadAll(@Param('docNo') docNo: string, @UserId() userId: string) {
    // TODO: ตรวจ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.getSgiDocumentAttachmentsDownloadAll(docNo, userId);
  }
}
```

9.3 DTO + Validation

```
// src/modules/sgi-attachment-sales-timeline/dto/sgi-attachment-sales-timeline.dto.ts
import { Type } from 'class-transformer';
import {
  IsArray, IsBoolean, IsIn, IsInt, IsNotEmpty, IsNumber, IsObject, IsOptional,
  IsString, Matches, Max, MaxLength, Min, ValidateNested,
} from 'class-validator';

// ValidationPipe ระดับ global ตั้ง whitelist + forbidNonWhitelisted + transform ไว้แล้ว (main.ts)
// property ที่ไม่ประกาศที่นี่จะถูก reject เป็น 400 อัตโนมัติ

// body ของ POST /api/v1/sgi/document/{docNo}/attachments
export class CreateSgiDocumentAttachmentsBodyDto {
  /** validate extension and content type */
  // TODO: ใช้ FileInterceptor + ValidationPipe แยก ไม่ผ่าน class-validator
  file: Express.Multer.File;

  /** บันทึกว่าแนบในขั้นไหน */
  @IsNotEmpty()
  @IsString()
  sectionCode: string;
}
```

9.4 Service (inject `DATA_SOURCE` + raw SQL)

service ประกาศ method ครบทุกเส้นที่ controller เรียก และ signature มาจากแหล่งเดียวกับ controller (จำนวน/ลำดับพารามิเตอร์จึงตรงกันเสมอ) — เส้นที่ยังไม่ได้ implement เป็น stub ที่ `throw new NotImplementedException(...)` ให้ TypeScript compile ผ่านตั้งแต่วันแรก

```
// src/modules/sgi-attachment-sales-timeline/sgi-attachment-sales-timeline.service.ts
import { Inject, Injectable, Logger, NotImplementedException } from '@nestjs/common';
import { DataSource } from 'typeorm';
import { SgiException } from '../sgi/exceptions/sgi.exception';
import { SGI_SQL } from './sgi-attachment-sales-timeline.sql';

@Injectable()
export class SgiAttachmentSalesTimelineService {
  private readonly logger = new Logger(SgiAttachmentSalesTimelineService.name);

  constructor(
    // DATA_SOURCE override query(): SELECT/WITH ไป slave pool, write ไป master
    @Inject('DATA_SOURCE') private readonly dataSource: DataSource,
  ) {}

  // POST /api/v1/sgi/document/{docNo}/attachments — Upload attachment API
  // mutation ต้องอยู่ใน transaction เดียว (ไม่มี audit ของ master แล้ว · 2026-08-07)
  async createSgiDocumentAttachments(docNo: string, body: CreateSgiDocumentAttachmentsBodyDto, userId: string) {
    const runner = this.dataSource.createQueryRunner();
    await runner.connect();
    await runner.startTransaction();
    try {
      // TODO: lock แถวเป้าหมายของ sgi_document_attachments ด้วย SELECT ... FOR UPDATE ก่อนเขียน
      const [current] = await runner.query(SGI_SQL.createSgiDocumentAttachmentsLock, [docNo]);
      if (!current) {
        throw new SgiException('NOT_FOUND', 'ไม่พบข้อมูลที่ต้องการ');
      }
      await runner.query(SGI_SQL.createSgiDocumentAttachments, [/* TODO: ผูกค่าจาก body */]);
      await runner.commitTransaction();
      return { message: 'saved' };
    } catch (error) {
```

```
      await runner.rollbackTransaction();
      this.logger.error(error);
      throw error;
    } finally {
      await runner.release();
    }
  }

  // GET /api/v1/sgi/document/{docNo}/attachments/{attachId}/download — ดาวน์โหลดไฟล์แนบรายไฟล์ผ่าน BE — ตรวจสิทธิ์เอกสาร + attachment ต้องเป…
  async getSgiDocumentAttachmentsDownload(docNo: string, attachId: string, userId: string) {
    const page = 1;
    // DTO ของเส้นนี้ไม่มี page/size (ดูหัวข้อ DTO) — ไม่แบ่งหน้า
    const size = 100;
    // SQL เต็มอยู่ในหัวข้อ Database SQL ของเอกสารนี้ (คีย์ 'GET /api/v1/sgi/document/{docNo}/attachments/{attachId}/download')
    // SQL ในเอกสารเป็น positional $1..$n อยู่แล้ว (ตัวสร้างแปลงให้ตั้งแต่ 2026-09-04)
    // บรรทัดแรกของบล็อก SQL คือ `-- bind ตามลำดับ: $1=... · $2=...` ให้เรียงอาร์กิวเมนต์ตามนั้น
    const rows = await this.dataSource.query(SGI_SQL.getSgiDocumentAttachmentsDownload, [
      // เรียงให้ตรงกับบรรทัด `-- bind ตามลำดับ:` ของ SQL เส้นนี้
      userId, (page - 1) * size, size,
    ]);
    // TODO: total ต้องมาจาก COUNT(*) แยก query หรือ window function ไม่ใช่ rows.length
    return { page, size, total: rows.length, items: rows };
  }

  // GET /api/v1/sgi/document/{docNo}/attachments/download-all — ดาวน์โหลดไฟล์แนบทั้งหมดเป็น .zip — ไม่มีไฟล์ที่ผ่าน scan เลยตอบ 404 (…
  async getSgiDocumentAttachmentsDownloadAll(docNo: string, userId: string) {
    // TODO: implement ตาม business rule ของ GET /api/v1/sgi/document/{docNo}/attachments/download-all
    // (SQL อยู่ในหัวข้อ Database SQL คีย์ 'GET /api/v1/sgi/document/{docNo}/attachments/download-all')
    throw new NotImplementedException('getSgiDocumentAttachmentsDownloadAll ยังไม่ implement');
  }
}
```

9.5 Entity (TypeORM)

```
// src/entitys/sgi-document-attachments.entity.ts
import { Column, Entity, PrimaryColumn } from 'typeorm';

@Entity({ name: 'sgi_document_attachments', schema: process.env.DB_SCHEMA })
export class DocumentAttachment {
  @PrimaryColumn({ name: 'attach_id', type: 'bigint' })
  attachId: number;

  @Column({ name: 'doc_no', type: 'varchar', length: 10 })
  docNo: string;

  @Column({ name: 'section_code', type: 'varchar', length: 2 })
  sectionCode: string;

  @Column({ name: 'file_name', type: 'varchar', length: 255 })
  fileName: string;

  @Column({ name: 'mime_type', type: 'varchar', length: 100 })
  mimeType: string;

  @Column({ name: 'file_size', type: 'bigint' })
  fileSize: number;

  @Column({ name: 'storage_provider', type: 'varchar', length: 30 })
  storageProvider: string;

  @Column({ name: 'bucket', type: 'varchar', length: 120 })
  bucket: string;

  @Column({ name: 'object_key', type: 'varchar', length: 500 })
  objectKey: string;

  @Column({ name: 'sha256', type: 'varchar', length: 64 })
  sha256: string;

  @Column({ name: 'uploaded_by', type: 'varchar', length: 30 })
  uploadedBy: string;

  @Column({ name: 'uploaded_by_name', type: 'varchar', length: 200, nullable: true })
  uploadedByName?: string;

  @Column({ name: 'uploaded_at', type: 'timestamp' })
  uploadedAt: Date;

  @Column({ name: 'deleted_flag', type: 'char', length: 1, default: 'N' })
  deletedFlag: string;
```

```
 // entity ชุดนี้ generate จาก DDL ใน LLDD-Database.pdf §5.2–5.4 โดยตรง — คอลัมน์/ชนิด/nullable ตรงกันเสมอ
 // ไม่ประกาศ relation ตาม convention ของทีม (join ด้วย raw SQL)
ไ
// src/entitys/sgi-compensation-documents.entity.ts
import { Column, Entity, PrimaryColumn } from 'typeorm';

@Entity({ name: 'sgi_compensation_documents', schema: process.env.DB_SCHEMA })
export class CompensationDocument {
 @PrimaryColumn({ name: 'id', type: 'bigint' })
 id: number;

 @Column({ name: 'doc_no', type: 'varchar', length: 10, nullable: true })
 docNo?: string;

 @Column({ name: 'year', type: 'int', nullable: true })
 year?: number;

 @Column({ name: 'running_no', type: 'int', nullable: true })
 runningNo?: number;

 @Column({ name: 'impact_process_id', type: 'bigint' })
 impactProcessId: number;

 @Column({ name: 'impact_compensation_id', type: 'bigint' })
 impactCompensationId: number;

 @Column({ name: 'impacted_store_code', type: 'varchar', length: 5 })
 impactedStoreCode: string;

 @Column({ name: 'impact_month', type: 'char', length: 7, nullable: true })
 impactMonth?: string;

 @Column({ name: 'new_store_code', type: 'varchar', length: 5, nullable: true })
 newStoreCode?: string;

 @Column({ name: 'round_no', type: 'int', nullable: true })
 roundNo?: number;

 @Column({ name: 'loop_no', type: 'int', nullable: true })
 loopNo?: number;

 @Column({ name: 'source', type: 'varchar', length: 20, default: 'FS' })
 source: string;

 @Column({ name: 'status_code', type: 'varchar', length: 2, default: '06' })
 statusCode: string;

 @Column({ name: 'current_section_code', type: 'varchar', length: 2, nullable: true })
 currentSectionCode?: string;

 @Column({ name: 'total_compensation_amount', type: 'numeric', precision: 14, scale: 2, default: 0 })
 totalCompensationAmount: string;

 @Column({ name: 'allmap_url', type: 'varchar', length: 500, nullable: true })
 allmapUrl?: string;

 @Column({ name: 'statement_id', type: 'varchar', length: 500, nullable: true })
 statementId?: string;

 @Column({ name: 'statement_date', type: 'date', nullable: true })
 statementDate?: Date;

 @Column({ name: 'account_year', type: 'int', nullable: true })
 accountYear?: number;

 @Column({ name: 'account_month', type: 'int', nullable: true })
 accountMonth?: number;

 @Column({ name: 'approver_snapshot', type: 'jsonb', nullable: true })
 approverSnapshot?: Record<string, unknown>;

 @Column({ name: 'draft_result', type: 'varchar', length: 100, nullable: true })
 draftResult?: string;

 @Column({ name: 'draft_comment', type: 'text', nullable: true })
 draftComment?: string;

 @Column({ name: 'draft_section_code', type: 'varchar', length: 2, nullable: true })
 draftSectionCode?: string;

 @Column({ name: 'draft_by', type: 'varchar', length: 30, nullable: true })
 draftBy?: string;
```

```
  @Column({ name: 'draft_at', type: 'timestamp', nullable: true })
  draftAt?: Date;

  @Column({ name: 'version_no', type: 'int', default: 1 })
  versionNo: number;

  @Column({ name: 'created_by', type: 'varchar', length: 30 })
  createdBy: string;

  @Column({ name: 'created_at', type: 'timestamp' })
  createdAt: Date;

  @Column({ name: 'updated_by', type: 'varchar', length: 30, nullable: true })
  updatedBy?: string;

  @Column({ name: 'updated_at', type: 'timestamp', nullable: true })
  updatedAt?: Date;

  // entity ชุดนี้ generate จาก DDL ใน LLDD-Database.pdf §5.2–5.4 โดยตรง — คอลัมน์/ชนิด/nullable ตรงกันเสมอ
  // ไม่ประกาศ relation ตาม convention ของทีม (join ด้วย raw SQL)
}
```

ตารางที่เหลือของเอกสารนี้ (`sgi_fgi_impact_sales_summaries`, `sgi_sales_transactions`, `sgi_consideration_logs`) ใช้รูปแบบ entity เดียวกัน — คอลัมน์อ้างจาก `Database design`

9.6 Repository Providers + Module wiring

```
// src/providers/sgi/sgi.ts — repository provider แบบ factory (ไม่ใช้ TypeOrmModule.forFeature)
// convention ของโฟลเดอร์ providers คือ 1 ไฟล์ต่อโดเมน ตั้งชื่อตามโดเมน (business_user/business_user.ts,
// common_code/common_code.ts …) ไม่ใช่ index.ts
//
// ▲ ไฟล์นี้ใช้ร่วมกันทุกเอกสาร BE ของ SGI — ให้ **merge array เพิ่ม** เข้าไฟล์เดิม ห้ามเขียนทับ
// (ชื่อ const แยกต่อเอกสารไว้แล้วเพื่อไม่ให้ชนกัน)
import { DataSource } from 'typeorm';
import { DocumentAttachment } from '../../entitys/sgi-document-attachments.entity';
import { CompensationDocument } from '../../entitys/sgi-compensation-documents.entity';
import { FgiImpactSalesSummary } from '../../entitys/sgi-fgi-impact-sales-summaries.entity';

export const sgiAttachmentSalesTimelineProviders = [
  {
    provide: 'SGI_DOCUMENT_ATTACHMENT_REPOSITORY',
    useFactory: (dataSource: DataSource) => dataSource.getRepository(DocumentAttachment),
    inject: ['DATA_SOURCE'],
  },
  {
    provide: 'SGI_COMPENSATION_DOCUMENT_REPOSITORY',
    useFactory: (dataSource: DataSource) => dataSource.getRepository(CompensationDocument),
    inject: ['DATA_SOURCE'],
  },
  {
    provide: 'SGI_FGI_IMPACT_SALES_SUMMARIES_REPOSITORY',
    useFactory: (dataSource: DataSource) => dataSource.getRepository(FgiImpactSalesSummary),
    inject: ['DATA_SOURCE'],
  },
];

// src/modules/sgi-attachment-sales-timeline/sgi-attachment-sales-timeline.module.ts
import { MiddlewareConsumer, Module, NestModule } from '@nestjs/common';
import { DatabaseModule } from '../../database/database.module';
// UserContextMiddleware อ่าน header x-user-id แล้วเซ็ต request.userId ที่ @UserId() ใช้
// — app.module.ts **ไม่ได้** apply แบบ global (มีแค่ HttpContext/LoggerContext) แต่ละโมดูลต้อง apply เอง
// (ดู evaluation-process.module.ts / inform-evaluate.module.ts / cooperation-request.module.ts)
import { UserContextMiddleware } from '../../common/middleware/user-context.middleware';
import { sgiAttachmentSalesTimelineProviders } from '../../providers/sgi/sgi';
import { SgiAttachmentSalesTimelineController } from './sgi-attachment-sales-timeline.controller';
import { SgiAttachmentSalesTimelineService } from './sgi-attachment-sales-timeline.service';

@Module({
  imports: [DatabaseModule],
  controllers: [SgiAttachmentSalesTimelineController],
  providers: [SgiAttachmentSalesTimelineService, ...sgiAttachmentSalesTimelineProviders],
  exports: [SgiAttachmentSalesTimelineService],
})
export class SgiAttachmentSalesTimelineModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {
    // ถ้าไม่ apply ตรงนี้ userId จะเป็น undefined เงียบ ๆ ทุก endpoint
    consumer.apply(UserContextMiddleware).forRoutes(SgiAttachmentSalesTimelineController);
  }
}
// TODO: register module นี้ใน app.module.ts (imports) พร้อมกับโมดูล SGI ตัวอื่น
```

9.7 BFF Proxy (module + controller + client service)

BFF ยังไม่มีฟีเจอร์ประกันรายได้เลย จึงต้องสร้าง module ใหม่ + client service ใหม่ทั้งชุด และเลือก prefix แบบเดียวทั้งโมดูล (ที่นี่ใช้ /bff/sgi/…) เพื่อไม่ให้ปนแบบที่มี/ไม่มี `/bff` เหมือนโมดูลเดิม

```
// BFF — ไม่ต้องมี client ของ SGI แยก
//
// ● แก้ 2026-09-22 — skeleton เดิมเสนอ `SgiClientService` เป็น class แยกที่ *ชี้ไป store backend*
// ของจริงในรีโปกลับชี้ `config.api.sgi.url` (ค่าตั้งต้น http://localhost:3010)
// ซึ่ง **ไม่มี backend นั้นอยู่จริง** และ `API_SGI_BACKEND_URL` ไม่ได้ตั้งไว้ที่ไหนเลย
// → 12 เส้น (รวม 5 เส้นที่ FE เรียกจริง) ยิงไม่ถึงปลายทาง (พบตอนรันจริง)
// มติ DP-10 ระบุว่า SGI **ไม่แยก backend** — อยู่ใน srm-sps-spsap-store-backend
// จึงใช้ `StoreClientService` ที่มีอยู่แล้วตรง ๆ ไม่สราง client ตัวที่สอง
// (`tools/check_docs.py` มีกฎห้ามโมดูล SGI ของ BFF ใช้ SgiClientService)
//
// header: ใช้ `OperatingManagementService.header(req)` ชุดกลางเท่านั้น ห้ามประกอบเอง
// ตัวประกอบที่เขียนเองในโมดูลส่ง header ไม่ครบ (ขาด x-user-full-name / accept-language)
// และ serialize x-user-permissions คนละรูปแบบกับฝั่งรับที่ใช้จริง (JSON.parse)
//
// path ที่ forward ให้ใช้ `/api/v1/sgi/...` เสมอ — controller ฝั่ง BE บางตัวประกาศ base path
// เป็น array 2 ค่า แต่บางตัวมีเฉพาะ canonical
// BaseClientService และ { success, data } ให้แล้ว — service ฝั่ง BFF จึงได้ data ตรง ๆ
// src/modules/sgi-attachment-sales-timeline/sgi-attachment-sales-timeline.service.ts (BFF)
import { Injectable } from '@nestjs/common';
import { StoreClientService } from '@common/client-services/store-client.service';
import { OperatingManagementService } from '@modules/operating-management/operating-management.service';

@Injectable()
export class SgiAttachmentSalesTimelineBffService {
  constructor(
    private readonly client: StoreClientService,
    private readonly operatingManagementService: OperatingManagementService,
  ) {}

  // BFF ไม่มี DB — หน้าที่เดียวคือแนบ user context แล้ว forward
  // ▲ ต้อง unwrap envelope ของ store-backend 1 ชั้นก่อนคืน (ยืนยันจากโค้ดจริง 2026-09-04):
  // ResponseInterceptor ระดับ global ของ BFF ห่อผลลัพธ์เป็น { success, data, requestId } อีกที
  // ถ้าคืน { success, data } ดิบมา FE จะได้ data.data.data — BaseClientService จึงต้องคืน .data.data
  //
  // ● แก้ 2026-09-22 — เดิม skeleton ให้ประกอบ header เองด้วย `userHeaders(user)` 3 ตัว
  // ส่งไม่ครบตาม LLDD-BE-Integration-SBP-Platform.pdf §5.1 (ขาด x-user-full-name · accept-language)
  // และ serialize x-user-permissions เป็น comma-separated ขณะที่ฝั่งรับที่ใช้จริง
  // (OperatingManagementHeaderGuard.parsePermissions) ทำ JSON.parse — คนละรูปแบบกัน
  // ให้ใช้ตัวประกอบกลางตัวเดียวทั้งระบบ และรับ `req` ไม่ใช่ `req.user`
  async createSgiDocumentAttachments(docNo: string, body: any, req: any) {
    return this.client.post(`/api/v1/sgi/document/${docNo}/attachments`, body, { headers: await this.operatingManagementService.header(req) });
  }

  async getSgiDocumentAttachmentsDownload(docNo: string, attachId: string, params: any, req: any) {
    return this.client.get(`/api/v1/sgi/document/${docNo}/attachments/${attachId}/download`, { params, headers: await
this.operatingManagementService.header(req) });
  }

  async getSgiDocumentAttachmentsDownloadAll(docNo: string, params: any, req: any) {
    return this.client.get(`/api/v1/sgi/document/${docNo}/attachments/download-all`, { params, headers: await
this.operatingManagementService.header(req) });
  }
}

// ---------- src/modules/sgi-attachment-sales-timeline/sgi-attachment-sales-timeline.controller.ts (BFF) ----------
import { Body, Controller, Delete, Get, Param, Post, Put, Query, Req, UseGuards } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';

// path เดียวกับที่ FE เรียก (apiClient baseURL รวม /api/v1 แล้ว) — ห้ามตั้งตามชื่อเอกสาร LLDD
@Controller('sgi/document/:docNo/attachments')
@UseGuards(AuthGuard('jwt'))
export class SgiAttachmentSalesTimelineBffController {
  constructor(private readonly service: SgiAttachmentSalesTimelineBffService) {}

  // proxy ของ POST /api/v1/sgi/document/{docNo}/attachments
  @Post()
  createSgiDocumentAttachments(@Param('docNo') docNo: string, @Body() body: any, @Req() req: any) {
    return this.service.createSgiDocumentAttachments(docNo, body, req);
  }

  // proxy ของ GET /api/v1/sgi/document/{docNo}/attachments/{attachId}/download
  @Get(':attachId/download')
  getSgiDocumentAttachmentsDownload(@Param('docNo') docNo: string, @Param('attachId') attachId: string, @Query() query: any, @Req()
req: any) {
    return this.service.getSgiDocumentAttachmentsDownload(docNo, attachId, query, req);
  }
}
```

```
// TODO: register module ใน app.module.ts ของ BFF · imports ต้องมี ClientServiceModule + OperatingManagementModule
```

10. Database SQL

10.1 ตารางที่อ่าน/เขียน

Table / Object	R/W	Usage
sgi_document_attachments	R/W	metadata ไฟล์แนบและ section ที่แนบ
sgi_compensation_documents	R	ตรวจเอกสารและ impact_process_id
sgi_fgi_impact_sales_summaries	R	หัวข้อมูลยอดขาย growth_rate_diff/total_working_days
sgi_sales_transactions	R	ยอดขายรายวัน 4 windows
sgi_consideration_logs	R	timeline/history

10.2 SQL จริงต่อ Endpoint

POST /api/v1/sgi/document/{docNo}/attachments — Upload attachment API

```
-- bind ตามลำดับ: $1=docNo · $2=sectionCode · $3=fileName · $4=mimeType · $5=fileSize · $6=storageProvider · $7=bucket · $8=objectKey · $9=sha256 · $10=userId · $11=userFullName · $12=now
-- ตรวจขนาด ≤ 5MB, sanitize filename, sha256 ก่อน commit metadata
-- ไม่มีการสแกนไวรัส (ถอดออก 2026-10-05) · uploaded_by_name = ชื่อจาก x-user-full-name
INSERT INTO sgi_document_attachments (doc_no, section_code, file_name, mime_type, file_size, storage_provider, bucket, object_key, sha256, uploaded_by, uploaded_by_name, uploaded_at)
VALUES ($1 /* docNo */, $2 /* sectionCode */, $3 /* fileName */, $4 /* mimeType */, $5 /* fileSize */, $6 /* storageProvider */, $7 /* bucket */, $8 /* objectKey */, $9 /* sha256 */, $10 /* userId */, $11 /* userFullName */, $12 /* now */);
```

GET /api/v1/sgi/document/{docNo}/attachments/{attachId}/download — ดาวน์โหลดไฟล์แนบรายไฟล์ผ่าน BE — ตรวจสิทธิ์เอกสาร + attachment ต้องเป็นของ docNo ตามนโยบาย (ดาวน์โหลดได้ทุกไฟ…

```
-- bind ตามลำดับ: $1=docNo · $2=attachId
-- ตรวจสิทธิ์อ่านเอกสาร + attachment ต้องเป็นของ docNo · ไม่มีการสแกนไวรัส (ถอดออก 2026-10-05)
SELECT attach_id, bucket, object_key, file_name, mime_type, file_size, storage_provider
FROM sgi_document_attachments
WHERE doc_no = $1 /* docNo */ AND attach_id = $2 /* attachId */
  AND deleted_flag = 'N'; -- ไฟล์ที่ถูกลบแบบ soft delete ห้ามดาวน์โหลด
```

GET /api/v1/sgi/document/{docNo}/attachments/download-all — ดาวน์โหลดไฟล์แนบทั้งหมดเป็น .zip — ไม่มีไฟล์ที่ผ่าน scan เลยตอบ 404 (ไม่คืน zip เปล่า) · ● แก้ 2026-09-30…

```
-- bind ตามลำดับ: $1=docNo
-- รวมไฟล์แนบทั้งหมดเป็น .zip — ตรวจสิทธิ์อ่านเอกสารก่อน · ไม่มีการสแกนไวรัส (ถอดออก 2026-10-05)
-- ไม่มีไฟล์ที่ดาวน์โหลดได้เลย -> 404 (ไม่คืน zip เปล่า)
-- U0001f534 ต้อง SELECT storage_provider ด้วย (2026-09-30) — ใช้คัดแถว K2-LEGACY + file_size = 0 ออกก่อนยิง S3
SELECT attach_id, bucket, object_key, file_name, mime_type, file_size, storage_provider
FROM sgi_document_attachments
WHERE doc_no = $1 /* docNo */
  AND deleted_flag = 'N' -- ไฟล์ที่ถูกลบแบบ soft delete ห้ามรวมลง zip
ORDER BY section_code, attach_id;
```

10.3 Index / Constraint ที่ควรมี (ข้อเสนอ)

Table	DDL ที่เสนอ	ที่มา / หมายเหตุ
sgi_document_attachments	CREATE INDEX idx_document_attachments_doc_no ON sgi_document_attachments (doc_no, section_code);	อนุมานจากเงื่อนไข query ที่เอกสารนี้ระบุ — สร้างพร้อมสคริปต์ deploy ของ SGI
sgi_compensation_documents	CREATE UNIQUE INDEX uk_compensation_documents_business ON sgi_compensation_documents (impacted_store_code, account_year, account_month, round_no);	อนุมานจากเงื่อนไข query ที่เอกสารนี้ระบุ — สร้างพร้อมสคริปต์ deploy ของ SGI
sgi_fgi_impact_sales_summaries	CREATE INDEX idx_fgi_impact_sales_summaries_process ON sgi_fgi_impact_sales_summaries (impact_process_id);	อนุมานจากเงื่อนไข query ที่เอกสารนี้ระบุ — สร้างพร้อมสคริปต์ deploy ของ SGI

Table	DDL ที่เสนอ	ที่มา / หมายเหตุ
sgi_sales_transactions	CREATE INDEX idx_sales_transactions_summary ON sgi_sales_transactions (sales_summary_id, window_no, txn_date);	อนุมานจากเงื่อนไข query ที่เอกสารนี้ระบุ — สร้างพร้อมสคริปต์ deploy ของ SGI
sgi_consideration_logs	CREATE INDEX idx_consideration_logs_doc_no ON sgi_consideration_logs (doc_no, action_datetime DESC);	อนุมานจากเงื่อนไข query ที่เอกสารนี้ระบุ — สร้างพร้อมสคริปต์ deploy ของ SGI

ทั้งหมดเป็น ข้อเสนอ ไม่ใช่ข้อกำหนดจาก ข้อกำหนดทางธุรกิจ — ให้ตรวจกับ `EXPLAIN ANALYZE` บนข้อมูลจริง และรวมเข้าไฟล์ `sql/deploy-sgi-*.sql` แบบ idempotent (`CREATE INDEX IF NOT EXISTS`) ตาม pattern ที่ทีมใช้อยู่

11. Processing Flow

Step	Description
1	Validate docNo/permission
2	Validate file size/type
3	Store file metadata
4	Load sales summary and transactions
5	Return timeline ordered by action time

12. Acceptance Criteria

- file >5MB returns 413
- unsupported file type returns 415
- sales windows are ordered
- timeline newest/oldest order matches FE expectation

13. Developer Test Checklist

No	Test
1	upload success
2	upload too large
3	download missing file
4	sales not found
5	timeline empty

14. Unit Test Scope

8 ชั่วโมง (30% ของ implementation 26 ชั่วโมง) · เครื่องมือ: Jest + mock repository/DataSource (ไม่ต่อ DB จริง)

หัวข้อนี้คือ unit test ที่ต้องเขียนคู่กับโค้ด — ต่างจาก *Developer Test Checklist* ซึ่งเป็น scenario ระดับ end-to-end/manual ที่ใช้ตอนตรวจรับ · รายการด้านล่าง derive จาก field/validation, acceptance criteria, endpoint และตารางที่เอกสารนี้เขียน

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
`docNo`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: required when opening existing document · รูปแบบ: YYYY/xxxxx
`storeCode`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: numeric length = 5 · รูปแบบ: string 5 digits

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
`amount`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: >= 0 · รูปแบบ: number, 2 decimals
`percent`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: 0-100 · รูปแบบ: number, 2 decimals
`sourceSystem`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: ALLMAP / USER · รูปแบบ: enum
`date`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: valid date · รูปแบบ: DD/MM/YYYY
`attachment`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: <= 5 MB · รูปแบบ: file
`file`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: <=5MB · รูปแบบ: multipart
`sectionCode`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: required on upload · รูปแบบ: string
business rule	logic	file >5MB returns 413
business rule	logic	unsupported file type returns 415
business rule	logic	sales windows are ordered
business rule	logic	timeline newest/oldest order matches FE expectation
`POST /api/v1/sgi/document/{docNo}/attachments`	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
`GET /api/v1/sgi/document/{docNo}/attachments/{attachId}/download`	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
`GET /api/v1/sgi/document/{docNo}/attachments/download-all`	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
`GET /api/v1/sgi/document/{docNo}/timeline`	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
`sgi_document_attachments`	transaction	จำลอง error กลางทาง แล้วยืนยันว่า rollback ครบ ไม่เหลือแถวค้าง (mock DataSource/QueryRunner)
service	error mapping	แปลง error ของ repository/lib เป็น error code ตามสัญญากลาง (LLDD-BE-API-Common-Contracts.pdf)

- ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง — mock ที่ขอบ repository/client เสมอ
- ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่
- เกณฑ์ผ่านของ CI: ทุกเคสในตารางนี้มี test จริงและผ่านทั้งหมด